BingoClip

Bingo cooperativo de videogame com clips automáticos

Documento de arquitetura e plano de viabilidade — MVP de custo zero

Versão 1.0 · 10/10/2026 · Autor: Pedro · Nome do projeto provisório

Valores de planos gratuitos conferidos em 10/10/2026 em páginas públicas; revise antes do lançamento.

Sumário

1. Resumo executivo 2
2. Objetivos, escopo e requisitos 2
3. Princípios de projeto 3
4. Pesquisa de viabilidade e limites gratuitos 4
5. Visão geral da arquitetura 5
6. Decisões de arquitetura (ADRs) 6
7. Experiência do novo usuário (onboarding) 7
8. Modelo de dados e regras no banco 9
9. Tempo real e sincronização 11
10. Pipeline de clips 11
11. App de desktop (Tauri v2) 12
12. Dados e analytics 13
13. Segurança e privacidade 13
14. Orçamento de custo zero e capacidade 14
15. Riscos e spikes de validação 15
16. Plano de implementação em fases 16
17. Engenharia: repositório, qualidade e entrega 16
18. Backlog inicial do MVP 17
19. Glossário e próximos passos 18
20. Fontes consultadas (10/10/2026) 19

1. Resumo executivo

O **BingoClip** é um bingo de videogame cooperativo. Um grupo de amigos entra numa sala, recebe o mesmo tabuleiro com objetivos de jogos (derrotar um chefe, coletar madeira, conversar com um NPC) e tenta completar um número de linhas antes de o tempo acabar, cada um jogando o seu jogo. Quando alguém marca uma casa, o app salva automaticamente os últimos segundos de gameplay (via OBS) e o clip aparece na sala. No fim, o sistema monta um vídeo com os melhores momentos da partida.

Veredito de viabilidade: é viável construir o MVP com **custo zero** para grupos pequenos, usando apenas planos gratuitos. As três decisões que sustentam isso são:

- **Supabase como backend único** (banco, login, tempo real e armazenamento). Não existe servidor próprio, VPN nem túnel: todos os clientes só fazem conexões de saída para a internet e funcionam atrás de qualquer roteador.
- **Uma interface, duas cascas.** No navegador o convidado joga o bingo sem instalar nada. Quem quer clips instala um app de desktop leve (Tauri) que traz a mesma interface mais a ponte com o OBS.
- **Clips pequenos e efêmeros** (720p, 20 s, cerca de 7,5 MB), apagados automaticamente, para caber no 1 GB de armazenamento gratuito.

Principais riscos, todos com mitigação neste documento: o atrito de configurar o OBS, o limite de tráfego de saída do armazenamento gratuito (5 GB por mês), a pausa do projeto gratuito após 1 semana sem atividade e o aviso do Windows SmartScreen enquanto o instalador não for assinado.

Valor para o portfólio: o projeto exercita backend com regras no banco, tempo real, pipeline de eventos, camada analítica em SQL, mídia (ffmpeg), app de desktop, CI/CD e distribuição, que é exatamente a mistura de engenharia de dados e backend Python/SQL das vagas que você está buscando.

2. Objetivos, escopo e requisitos

2.1 Objetivos

- O1. Jogar bingo cooperativo em tempo real com amigos, com um link de convite.
- O2. Gerar clips automáticos no momento em que uma casa é marcada, sem ninguém apertar atalho.
- O3. Entregar um vídeo de highlights ao final da partida.
- O4. Gerar estatísticas da noite (casas mais difíceis, tempo por objetivo, quem marcou mais).
- O5. Custo financeiro zero e instalação simples para quem nunca viu o projeto.

2.2 Escopo do MVP

- Salas privadas com código, modo cooperativo, tabuleiros 3×3, 4×4 e 5×5 (sem casa livre).

- Biblioteca inicial de objetivos genéricos e por jogo, com objetivos personalizados digitados pelo anfitrião.
- Login anônimo com apelido, com opção de vincular ao Discord.
- App de desktop para Windows com assistente de configuração do OBS, clip por marcação e envio automático.
- Galeria de clips na sala e vídeo final montado localmente.
- Limpeza automática de clips e dados antigos.

2.3 Fora do escopo do MVP

- Modo versus entre equipes, anti-bingo e multi-bingo (Fase 5).
- Marcação automática por conquistas da Steam ou verificação do clip por modelo de visão (Fase 5).
- Captura de tela embutida, sem OBS (futuro).
- Versões macOS e Linux do app (o código Tauri permite, mas não é prioridade do MVP).
- Qualquer funcionalidade paga, publicidade ou moderação de conteúdo público.

2.4 Requisitos não funcionais (metas a validar)

Requisito	Meta
Custo mensal	R$ 0 dentro dos limites gratuitos descritos na seção 14.
Entrada de convidado	No máximo 60 s e 3 toques, sem instalar nada.
Ativar clips	No máximo 10 min, incluindo instalar o app e configurar o OBS.
Latência de marcação	Casa marcada aparece para todos em até 1 s (rede doméstica).
Latência do clip	Clip visível para todos em até 10 s após a marcação.
Consumo do app	RAM do app em repouso abaixo de 100 MB; o peso real fica no encoder do OBS.
Disponibilidade	Melhor esforço. Pode haver pausa do plano gratuito, mitigada na seção 14.

3. Princípios de projeto

1. **Custo zero antes de escala.** Cada decisão é avaliada primeiro pelo limite gratuito que a sustenta.
2. **Quem só quer jogar entra sem instalar nada.** O app de desktop é opcional e adiciona capacidades.
3. **Capacidades progressivas.** Sem OBS, o bingo funciona; com OBS, aparecem os clips.
4. **O banco é a fonte da verdade.** As regras do jogo vivem no Postgres (RPC), e o cliente nunca é confiável.
5. **Efêmero por padrão.** Vídeo e dados brutos têm prazo de validade, por privacidade e por espaço.

6. **Decidir com evidência.** Toda suposição técnica crítica vira um spike curto antes de virar código.

4. Pesquisa de viabilidade e limites gratuitos

A tabela resume o que cada peça entrega no plano gratuito, com base nas páginas públicas dos provedores. Planos gratuitos mudam; por isso as decisões foram tomadas com folga e com plano B.

Peça	Uso no projeto	Limite gratuito	Observações
Supabase Postgres	Estado do jogo, perfis, eventos	500 MB por projeto; 2 projetos ativos	Projeto pausa após 1 semana sem atividade no banco.
Supabase Auth	Login anônimo e Discord	50.000 usuários ativos/mês	Login anônimo e OAuth social entram no plano gratuito.
Supabase Realtime	Sincronizar a sala	200 conexões; 100 msg/s; presença 20 msg/s	Payload de broadcast até 256 KB; limites por projeto.
Supabase Storage	Clips efêmeros	1 GB; 50 MB por arquivo	Egress de 5 GB/mês; sem redimensionamento de imagem.
Supabase Edge Functions	Limpeza de clips	500 mil invocações/mês	Opcional; usada na limpeza.
Cloudflare Pages	Hospedar a UI web	500 builds/mês; banda ilimitada; 25 MiB por arquivo	Domínio gratuito *.pages.dev.
Cloudflare R2 (plano B)	Clips, se o Storage apertar	10 GB; 1 M ops classe A; 10 M ops classe B	Saída de dados gratuita; exige um pequeno emissor de URLs assinadas.
GitHub	Código, Releases, CI	Actions ilimitado em repositório público	Privado: 2.000 min/mês (Windows conta o dobro).
SignPath Foundation	Assinar o instalador Windows	Gratuito para projetos de código aberto	Exige aprovação; um relato indica cerca de 1 semana.
OBS Studio	Captura de clips	Gratuito; WebSocket embutido desde o OBS 28	Porta padrão 4455, com senha.
ffmpeg	Montar o vídeo final	Gratuito	Usar build LGPL como processo separado; confira a licença.

4.1 Alternativas avaliadas e descartadas por enquanto

Alternativa	Por que não agora
Servidor no PC do anfitrião + túnel gratuito da Cloudflare	A sala morre se o anfitrião cair, e os túneis rápidos são para desenvolvimento, sem garantia de serviço.
VPN virtual (Hamachi, Radmin, Tailscale)	Cada amigo teria de instalar e entrar numa rede. Os dados do jogo são minúsculos, então é exagero. O Tailscale gratuito cobre 6 usuários, mas o atrito de entrada continua.
WebRTC peer-to-peer	Servidores STUN públicos ajudam, mas redes restritivas exigem um servidor TURN, e a complexidade sobe muito.
Electron no lugar do Tauri	Instaladores típicos de 80–150 MB e 150–300 MB de RAM em repouso, contra 2–10 MB e 30–50 MB no Tauri (valores de artigos de comparação, a confirmar no nosso app).
Firebase	Firestore é NoSQL; as estatísticas e as regras relacionais ficam mais naturais em SQL. O Postgres também treina o que as vagas pedem.

5. Visão geral da arquitetura

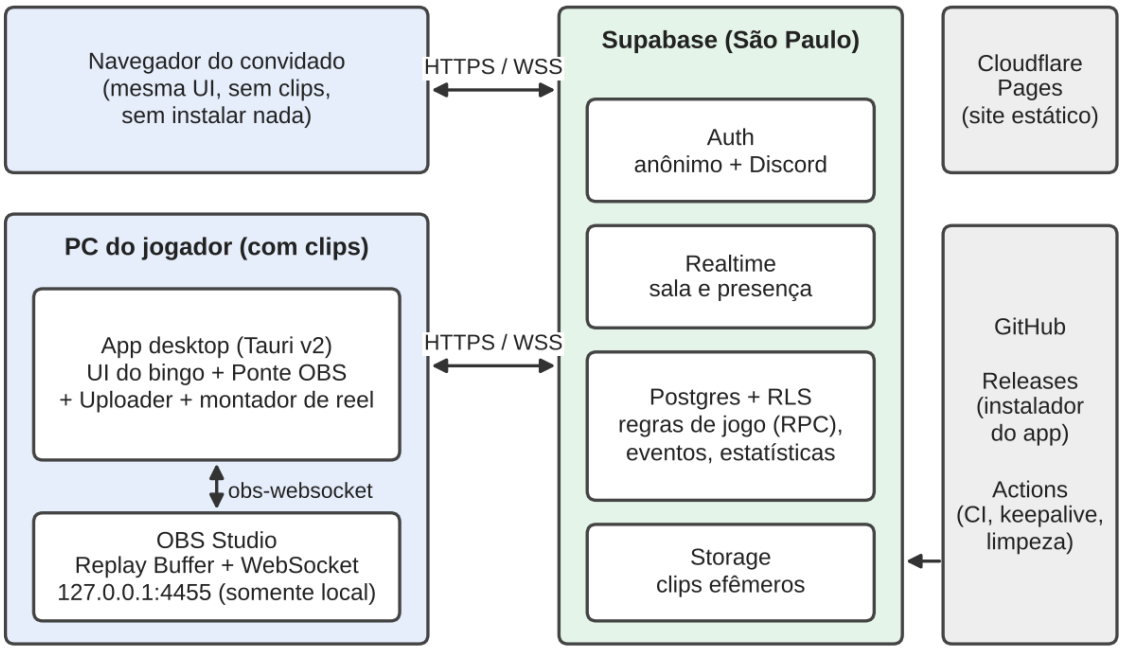


Figura 1 — Componentes do sistema. Todas as setas partem do cliente: não há portas abertas.

- **C1. Cliente web** (React + Vite + TypeScript), publicado no Cloudflare Pages. É o que o convidado abre pelo link.
- **C2. App de desktop** (Tauri v2): a mesma interface, com módulos nativos para ponte com o OBS, envio de clips, montagem do vídeo final, atualização e deep link.
- **C3. Supabase**, em região de São Paulo para reduzir a latência: Auth, Postgres com RLS e RPCs, Realtime e Storage.

- **C4. OBS Studio** local, com o WebSocket acessível só em 127.0.0.1. Nunca é exposto à rede.
- **C5. GitHub**: repositório público, Actions (CI, release, keepalive e limpeza) e Releases (distribuição do instalador).
- **C6. Camada analítica** (Fase 4): views e materialized views no próprio Postgres, com painel dentro do app.

5.1 Fluxo de uma marcação com clip

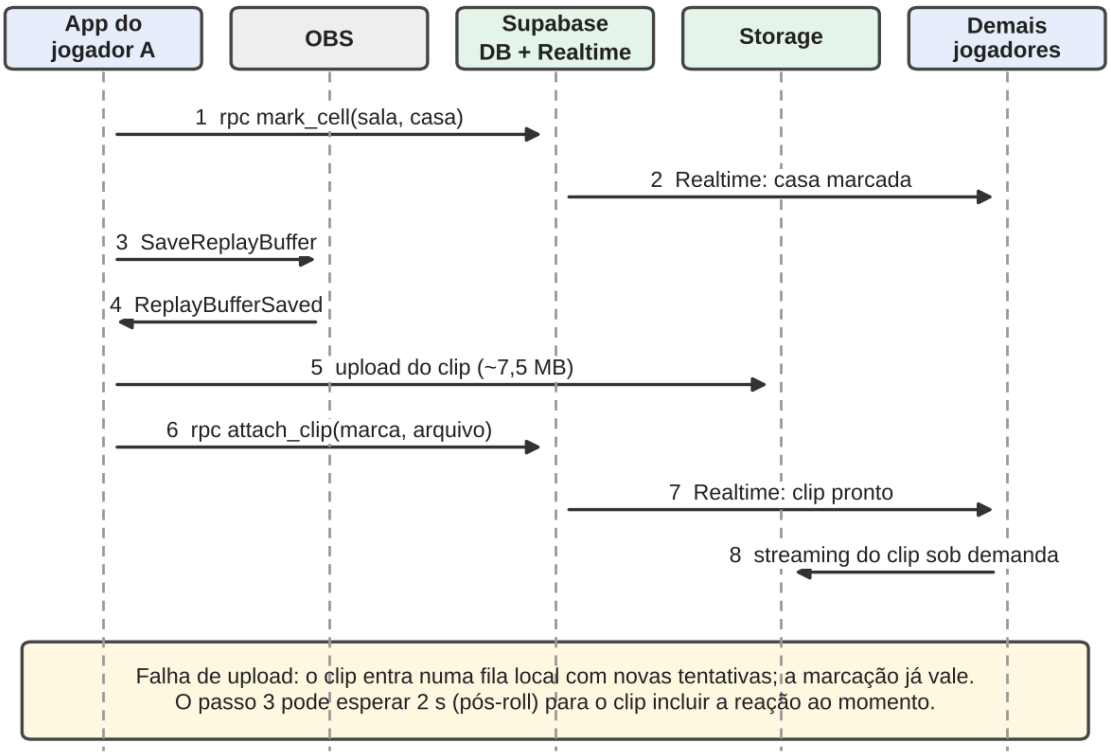


Figura 2 — Sequência de uma marcação. A marcação não depende do clip.

Dois pontos importantes: a marcação (passos 1 e 2) é independente do clip (passos 3 a 7), então uma falha de upload nunca bloqueia o jogo; e o clip é pedido ao OBS logo após o clique, porque o Replay Buffer guarda os segundos que *antecederam* o momento.

6. Decisões de arquitetura (ADRs)

Cada decisão abaixo é reversível. Quando o texto diz “consequência”, é o preço que se aceita no MVP.

ID	Decisão	Alternativa	Motivo e consequência
ADR-01	Supabase como backend único	Servidor próprio + túnel; Firebase	Zero hospedagem, nada exposto, SQL para analytics. Consequência: dependência do plano gratuito (pausa, limites).

ID	Decisão	Alternativa	Motivo e consequência
ADR-02	Uma UI, duas cascas (web + desktop)	Só web; só desktop	Convidado joga sem instalar. Clips exigem app local, porque o navegador não lê o arquivo que o OBS grava no disco.
ADR-03	Tauri v2 com lógica em TypeScript	Electron	Instalador e RAM menores. A curva do Rust fica restrita ao shell. Se faltar um plugin, o fallback é Electron.
ADR-04	OBS via obs-websocket	Captura própria (Windows Graphics Capture)	O OBS já traz o Replay Buffer pronto. Consequência: dependência externa, tratada com assistente. Modo sem OBS fica para o futuro.
ADR-05	Regras no Postgres (RPC security definer)	Lógica no cliente; API em Node/Python	Primeiro a marcar vence sem condição de corrida, cliente não confiável, testes com pgTAP.
ADR-06	Login anônimo + upgrade para Discord	E-mail e senha obrigatórios	Zero atrito para convidados; gamers já têm Discord. O vínculo preserva o histórico do mesmo usuário.
ADR-07	Clips de 720p e 20 s no Supabase Storage	R2 desde o início	Uma conta a menos para começar. Migra para o R2 se o tráfego de saída apertar.
ADR-08	Vídeo final montado localmente com ffmpeg	Montar no servidor	Sem custo de computação. Só o editor da sala baixa os clips selecionados.
ADR-09	Realtime por Postgres Changes no MVP	Broadcast puro	Mais simples e consistente com RLS. Revisar para broadcast a partir do banco se a escala exigir (validar na documentação).

7. Experiência do novo usuário (onboarding)

Este é o ponto que decide se os seus amigos vão usar. O desenho separa quem só quer jogar de quem quer clips.

7.1 Jornada A — convidado, sem instalar (meta: 60 s, 3 toques)

1. Você cria a sala e copia o link de convite, por exemplo `https://bingoclip.pages.dev/j/K7QF2M` (endereço ilustrativo).
2. O amigo abre o link no celular ou no PC. A página já mostra o nome da sala e quem está dentro.
3. Ele digita um apelido e toca em **Entrar**. Por baixo, o site faz um login anônimo, sem e-mail e sem senha.
4. Cai no lobby, vê o tabuleiro e joga. Sem clips, mas a pontuação funciona normalmente.

7.2 Jornada B — ativar clips (meta: 10 min)

1. Na sala aparece um aviso discreto: “Quer clips automáticos? Instale o app”. O botão detecta o sistema e aponta para o instalador mais recente nas Releases do GitHub.
2. Instalação em um clique (instalador gerado pelo Tauri). Enquanto não houver assinatura, o Windows mostra o aviso do SmartScreen. A página de download traz o passo a passo (Mais informações, depois Executar assim mesmo) e o hash SHA-256.
3. Ao abrir, o app recebe o código da sala por deep link (`bingoclip://j/K7QF2M`), faz o próprio login anônimo e entra na mesma sala. O app passa a substituir a aba do navegador.
4. Um assistente faz quatro verificações: (a) OBS instalado e aberto; (b) WebSocket acessível, pedindo a senha uma vez ou lendo a configuração do OBS, se o spike S1 confirmar; (c) Replay Buffer ativo, ligado por comando e, se a saída do OBS estiver desativada, com passo a passo ilustrado; (d) clip de teste de 5 s gravado, com miniatura.
5. Pronto: o selo “clips ativos” aparece ao lado do apelido para toda a sala.

7.3 Conta e persistência

O padrão é o login anônimo. O botão **Salvar meu histórico** vincula a identidade ao Discord (OAuth), mantendo o mesmo usuário, para acumular estatísticas ao longo do tempo. E-mail e senha só entram se for realmente necessário.

7.4 Fricções previstas e respostas

Fricção	Resposta
Não tenho OBS	O modo sem clips funciona. O assistente aponta o site do OBS. Futuro: captura embutida.
O Windows disse que é perigoso	Página explica o aviso, mostra o SHA-256, build feita pelo GitHub Actions; pedir a assinatura gratuita da SignPath.
Não sei ligar o WebSocket	Assistente guiado; tentar autoconfigurar quando possível.
Meu PC é fraco	Preset 720p a 30 fps, encoder de hardware e buffer de 20 s.
Jogo em tela cheia fica preto	Orientar modo janela sem bordas e Captura de Jogo; checklist por jogo.
Perdi o link da sala	Código de 6 caracteres e lista de salas recentes no armazenamento local.
Meu clip falhou	Fila local com reenvio automático e botão manual; a marcação continua válida.

8. Modelo de dados e regras no banco

O esquema é pequeno e relacional. Toda escrita do cliente passa por funções RPC; as tabelas não têm políticas de INSERT, UPDATE ou DELETE para o cliente. A seguir, a versão simplificada das migrations.

8.1 Tabelas

```
create table profiles (
  id uuid primary key references auth.users on delete cascade,
  nickname text not null check (char_length(nickname) between 2 and 24),
  created_at timestamptz not null default now());

create table games (id serial primary key, slug text unique not null, name text not null);

create table goal_templates (
  id uuid primary key default gen_random_uuid(),
  game_id int references games, text text not null,
  kind text check (kind in ('boss','collect','npc','explore','combat','misc')),
  difficulty smallint check (difficulty between 1 and 5),
  created_by uuid references profiles, is_public boolean not null default false);

create table rooms (
  id uuid primary key default gen_random_uuid(),
  code text unique not null,                        -- 6 caracteres, sem 0/O/1/I
  host_id uuid not null references profiles,
  size smallint not null default 4 check (size in (3,4,5)),
  status text not null default 'lobby'
    check (status in ('lobby','running','finished','expired')),
  seed bigint not null, duration_s int not null default 3600,
  target_lines smallint not null default 2,
  clip_policy text not null default 'optional'
    check (clip_policy in ('off','optional','required')),
  started_at timestamptz, ends_at timestamptz,
  expires_at timestamptz not null default now() + interval '48 hours');

create table room_members (
  room_id uuid references rooms on delete cascade,
  user_id uuid references profiles,
  clips_active boolean not null default false,
  primary key (room_id, user_id));

create table cells (                                -- tabuleiro congelado da sala
  room_id uuid references rooms on delete cascade, idx smallint,
  goal_text text not null, game_slug text, kind text,
  primary key (room_id, idx));

create table marks (
  id uuid primary key default gen_random_uuid(),
  room_id uuid not null, cell_idx smallint not null,
  user_id uuid not null references profiles,
  marked_at timestamptz not null default now(),
  unique (room_id, cell_idx),                       -- primeiro a marcar vence
  foreign key (room_id, cell_idx) references cells (room_id, idx) on delete cascade);

create table clips (
  id uuid primary key default gen_random_uuid(),
  mark_id uuid unique references marks on delete cascade,
  room_id uuid not null, user_id uuid not null,
  storage_path text not null, bytes int, duration_ms int, thumb_path text,
  expires_at timestamptz not null default now() + interval '48 hours');

create table events (                               -- log append-only para analytics
  id bigint generated always as identity primary key,
  room_id uuid, user_id uuid, type text not null,
  payload jsonb not null default '{}', at timestamptz not null default now());
```

```
create index on events (room_id, at);
```

8.2 Segurança por linha (RLS)

```
create function is_member(p_room uuid) returns boolean
language sql stable security definer set search_path = public as $$
  select exists (select 1 from room_members
                 where room_id = p_room and user_id = auth.uid());
$$;

alter table rooms enable row level security;   -- idem: room_members, cells,
alter table marks enable row level security;    -- marks, clips, events

create policy rooms_read on rooms for select using (is_member(id));
create policy marks_read on marks for select using (is_member(room_id));
create policy clips_read on clips for select using (is_member(room_id));
-- sem policies de escrita: tudo passa pelas RPCs abaixo
```

8.3 Regras de jogo (RPCs)

Funções expostas ao cliente: `create_room`, `join_room(code)` (necessária porque quem ainda não é membro não consegue ler a sala), `start_room`, `mark_cell`, `attach_clip`, `server_time` e `ping`. O núcleo do jogo é `mark_cell`:

```
create function mark_cell(p_room uuid, p_idx smallint) returns marks
language plpgsql security definer set search_path = public as $$
declare r rooms; m marks;
begin
  select * into r from rooms where id = p_room;
  if not is_member(p_room) then raise exception 'not_member'; end if;
  if r.status <> 'running' or now() > r.ends_at then
    raise exception 'room_not_running';
  end if;
  begin
    insert into marks (room_id, cell_idx, user_id)
    values (p_room, p_idx, auth.uid()) returning * into m;
  exception when unique_violation then
    raise exception 'already_marked';
  end;
  insert into events (room_id, user_id, type, payload)
  values (p_room, auth.uid(), 'mark', jsonb_build_object('cell', p_idx));
  perform check_win(p_room);  -- conta linhas completas; se >= target_lines, finaliza
  return m;
end $$;
```

- **Geração do tabuleiro:** `start_room` sorteia `size*size` objetivos da biblioteca de forma determinística pelo `seed` (ordenando por `md5(seed || id)`), grava em `cells` e define `started_at` e `ends_at`. Todos veem o mesmo tabuleiro.
- **Vitória:** `check_win` considera linhas, colunas e as duas diagonais (3×3 tem 8 linhas, 4×4 tem 10, 5×5 tem 12). Quando as linhas completas atingem `target_lines`, a sala vira `finished`.
- **Limites:** `attach_clip` recusa mais de 10 clips por jogador na partida; `mark_cell` aplica um limite simples de marcações por minuto usando `events`.
- **Códigos de sala:** 6 caracteres de um alfabeto sem ambiguidade (cerca de 1 bilhão de combinações), com limite de tentativas de entrada por usuário.

8.4 Armazenamento de clips

```
-- caminho: clips/{room_id}/{user_id}/{mark_id}.mp4
create policy members_read_clips on storage.objects for select
  using (bucket_id = 'clips'
         and public.is_member((storage.foldername(name))[1]::uuid));

create policy members_upload_own_clips on storage.objects for insert
  with check (bucket_id = 'clips'
         and public.is_member((storage.foldername(name))[1]::uuid)
         and (storage.foldername(name))[2] = auth.uid()::text);
```

9. Tempo real e sincronização

- **Canal por sala.** O cliente assina mudanças em `marks`, `clips`, `rooms` e `room_members` filtradas por `room_id`, mais Presence para “online” e “clips ativos”. Evitar atualizar presença com frequência, por causa do limite de 20 msg/s no plano gratuito.
- **Estados da sala:** lobby, running, finished e expired. Só o anfitrião inicia a partida.
- **Relógio.** O servidor define `ends_at`. O cliente mede o desvio do relógio chamando `server_time()` ao entrar e mostra a contagem regressiva localmente. A RPC recusa marcações fora do tempo.
- **Marcação otimista.** A interface pinta a casa na hora. Se a RPC devolver `already_marked`, desfaz e mostra quem marcou primeiro.
- **Reconexão.** O cliente reconecta sozinho e, ao voltar, recarrega o instantâneo da sala (cells, marks, clips). O banco é a verdade, então não há reconciliação complexa.
- **Idempotência.** A restrição `unique (room_id, cell_idx)` impede marcações duplicadas.

10. Pipeline de clips

10.1 Parâmetros recomendados

Parâmetro	Valor padrão	Observação
Duração do Replay Buffer	20 s	Guarda os últimos 20 s antes do clique.
Pós-roll	2 s	Espera antes de salvar, para incluir a reação.
Resolução e quadros	1280×720 a 30 fps	Configurar na saída do OBS (confirmar no spike S1).
Taxa de bits	3 Mbps (CBR)	3 Mbps × 20 s = 60 Mbit, cerca de 7,5 MB por clip.
Clips por jogador	até 10 por partida	Aplicado em attach_clip.
Retenção	24 h após o fim da sala	Limpeza por Actions e Edge Function.

No CBR, o tamanho do buffer em memória é a taxa de bits vezes o tempo, então 20 s a 3 Mbps ocupam poucos MB de RAM. Segundo a documentação da comunidade, o custo relevante é o do encoder, e por isso o preset de 720p a 30 fps, preferindo o

encoder de hardware.

10.2 Etapas

1. O jogador toca na casa. O app chama `mark_cell` e, em paralelo, agenda o salvamento do replay após o pós-roll.
2. O app envia `SaveReplayBuffer` ao OBS e espera o evento `ReplayBufferSaved`, que informa o caminho do arquivo (a confirmar no spike S1; alternativa: `GetLastReplayBufferReplay`).
3. O app valida que o arquivo está dentro da pasta de gravação informada pelo OBS (`GetRecordDirectory`) e normaliza se necessário.
4. Envia para o Storage no caminho da seção 8.4, com tentativas exponenciais em caso de falha. Itens pendentes ficam numa fila persistida em disco.
5. Chama `attach_clip`. O Realtime avisa a sala e as miniaturas ganham o botão de reproduzir.
6. Ao final da partida, o editor da sala escolhe os clips das linhas vencedoras (e alguns extras), baixa só esses e gera o vídeo.

10.3 Limpeza e privacidade

- Um workflow do GitHub Actions roda a cada 12 h: chama `ping` (mantém o projeto ativo), a Edge Function `purge-expired` (apaga objetos vencidos pela API do Storage e depois as linhas) e `refresh_stats`.
- Evitar apagar objetos do Storage por SQL; usar a API, para não deixar arquivos órfãos.
- O jogador pode apagar os próprios clips a qualquer momento, e um clip só é enviado se a política da sala permitir.

11. App de desktop (Tauri v2)

A UI é a mesma do site. O shell nativo expõe poucos comandos e módulos em TypeScript sempre que possível.

Módulo	Responsabilidade	Detalhes
obs	Conectar ao OBS e comandar o Replay Buffer	Cliente obs-websocket v5 em 127.0.0.1:4455; reconexão automática; comandos StartReplayBuffer, SaveReplayBuffer, GetRecordDirectory; evento ReplayBufferSaved.
uploader	Enviar clips ao Storage	Fila persistida, retentativas exponenciais, limite de 50 MB por arquivo, chamada a attach_clip.
reel	Montar o vídeo final	ffmpeg como sidecar baixado sob demanda; normaliza, concatena e escreve legenda com a casa e o apelido; salva em Vídeos.
updater	Atualização automática	Plugin oficial lendo o latest.json das Releases; pacotes assinados com chave própria.

Módulo	Responsabilidade	Detalhes
deeplink	Abrir a sala pelo link	Esquema bingoclip://j/CODIGO; valida o formato do código e nunca carrega tokens.
settings	Preferências	Duração do buffer, política de clips, porta e senha do OBS no armazenamento local do usuário.

- **Segurança:** permissões mínimas por capability do Tauri, CSP restritiva, leitura de arquivos só dentro da pasta de gravação do OBS, senha do OBS nunca enviada ao servidor.
- **Desempenho:** sem polling constante; o app é dirigido por eventos do OBS e do Realtime. A meta de RAM está na seção 2.4.
- **Pré-requisito do usuário:** Windows 10/11 (o WebView2 vem no Windows 11 e o instalador cuida do caso contrário, a validar).

12. Dados e analytics

Cada ação relevante vira uma linha em `events` (append-only). Estatísticas são views no próprio Postgres, atualizadas pelo workflow de manutenção. Isso é útil de verdade e também demonstra engenharia de dados no portfólio.

```
create materialized view stats_cell_difficulty as
select c.goal_text, c.game_slug,
       count(m.id) as vezes_marcada,
       avg(extract(epoch from (m.marked_at - r.started_at))) as seg_medios_ate_marcar
from cells c
join rooms r on r.id = c.room_id
left join marks m on m.room_id = c.room_id and m.cell_idx = c.idx
group by 1, 2;
```

- **Estatísticas da noite:** casas mais difíceis, tempo até cada marcação, contribuição por jogador, linha vencedora.
- **Histórico:** partidas por usuário vinculado ao Discord, recordes por jogo.
- **Retenção:** eventos brutos por 90 dias; depois, só agregados (cabe com folga nos 500 MB).
- **Evolução (opcional):** exportar eventos para Parquet e analisar com DuckDB ou notebook; dbt-core local sobre o Postgres; dashboard em Metabase local. Nada disso gera custo.

13. Segurança e privacidade

- **RLS em todas as tabelas** e escrita só por RPC. A chave anônima do Supabase é pública por desenho; a chave de serviço fica apenas em segredos do GitHub e nunca vai ao cliente.
- **OBS WebSocket** só em localhost, sempre com senha. O app nunca abre portas.
- **Clips privados:** só membros da sala leem; acesso por URLs assinadas com validade curta; apagados automaticamente.

- **Captura de jogo, não da área de trabalho:** orientar o uso de Captura de Jogo para não gravar notificações ou conversas. Opção de pré-visualizar antes de enviar.
- **Dados pessoais mínimos:** apelido e, se o usuário vincular, os dados básicos do Discord. Aviso de privacidade curto (LGPD) com pedido de exclusão pelo próprio app.
- **Abuso:** limite de tentativas ao entrar por código, limite de marcações por minuto e CAPTCHA opcional no login anônimo.
- **Cadeia de entrega:** instalador construído pelo Actions a partir da tag, hash SHA-256 publicado, atualizações assinadas.

14. Orçamento de custo zero e capacidade

Estimativas conservadoras, baseadas em uma partida típica: 4 jogadores, até 10 clips cada, clips de cerca de 7,5 MB.

Recurso	Limite	Consumo estimado por partida	Capacidade aproximada
Storage	1 GB	40 clips × 7,5 MB = 300 MB (apagados em 24 h)	2 a 3 partidas simultâneas retidas
Egress	5 GB/mês	Galeria vista pelos 4 (~480 MB) + editor (~75 MB) ≈ 550 MB	Cerca de 9 partidas por mês
Banco	500 MB	~100 eventos × ~300 B ≈ 30 KB	Milhares de partidas; limpeza em 90 dias
Realtime	200 conexões; 100 msg/s	Marcações raras, menos de 1 msg/s por sala	Cerca de 50 salas simultâneas de 4
Pages	500 builds/mês	Só em deploys	Folga ampla
Actions	Ilimitado (público)	Keepalive 2×/dia, CI, releases	Folga ampla

14.1 O que vai quebrar primeiro, e o plano B

Gargalo	Sinal	Plano B (ainda grátis)
Egress do Storage (5 GB/mês)	Painel do Supabase perto do limite	Mover clips para o R2 (saída gratuita) com um Worker que emite URLs assinadas após validar o JWT.
Armazenamento (1 GB)	Uploads falhando	Reduzir bitrate para 2,5 Mbps ou clips para 15 s; limite de 8 clips; R2.
Pausa por inatividade	Projeto pausado após 7 dias	Workflow de keepalive; reativar no painel se acontecer.
Workflows agendados	Desativados em repositório sem atividade por 60 dias (comportamento do GitHub, a validar)	Commit periódico ou reativação manual.

15. Riscos e spikes de validação

15.1 Riscos

Risco	Prob.	Impacto	Mitigação
OBS difícil de configurar para amigos leigos	Alta	Alto	Assistente guiado, autoconfiguração onde possível, modo sem clips sempre disponível.
Jogo com tela preta ou anticheat reclamando da captura	Média	Médio	Lista de jogos testados, modo janela sem bordas, avisos no assistente.
Windows SmartScreen bloqueando o instalador	Alta	Médio	Página explicativa, SHA-256, assinatura gratuita da SignPath, projeto aberto.
Limite de egress ou storage do plano gratuito	Média	Médio	Clips pequenos, limpeza agressiva, plano B com R2.
Projeto Supabase pausado	Média	Médio	Keepalive agendado, aviso na página de status.
Mudança nos planos gratuitos	Média	Alto	Camadas desacopladas por ADR; tudo exportável; migrations no repositório.
Latência alta de tempo real	Baixa	Médio	Região de São Paulo, marcação otimista, snapshot na reconexão.
Curva do Rust no Tauri	Média	Baixo	Shell fino, lógica em TypeScript, fallback para Electron.

15.2 Spikes (rodar antes de construir)

ID	Pergunta	Critério de sucesso
S1	Um script salva o replay por tecla e descobre o caminho do arquivo? Dá para ler a senha/porta do OBS e ligar o Replay Buffer automaticamente?	Clip aparece em menos de 3 s; evento ReplayBufferSaved ou GetLastReplayBufferReplay entrega o caminho; decidir autoconfiguração ou assistente manual.
S2	O app Tauri conecta em ws://127.0.0.1:4455 sem bloqueio de CSP e lê o arquivo da pasta de gravação?	Conexão e leitura funcionando com permissões mínimas.
S3	A captura funciona nos jogos reais do grupo (tela cheia, janela sem bordas, anticheat)?	Pelo menos 3 jogos com clip válido.
S4	Latência do Realtime entre dois PCs na região de São Paulo.	Marcação visível em até 1 s em 95% dos casos.
S5	Tempo de upload de um clip de 7,5 MB em uma conexão doméstica.	Menos de 10 s até a sala ver o clip.
S6	Montagem de 10 clips com ffmpeg na sua máquina (com e sem encoder de hardware).	Vídeo pronto em menos de 2 min, sem travar o PC.
S7	Instalador do Tauri em um Windows limpo: WebView2, SmartScreen e atualização.	Instala e abre sem passos extras além do aviso documentado.
S8	Pedido de assinatura gratuita na SignPath Foundation.	Aprovação, ou plano de continuar sem assinatura documentado.

16. Plano de implementação em fases

As estimativas são de trabalho em meio período e incertas; servem para ordenar, não para prometer prazo.

Fase	Entrega	Pronto quando
0 — Fundação (1 fim de semana)	Repositório, dois projetos Supabase (dev e prod) em São Paulo, spikes S1, S2 e S4, CI básico.	Script salva clip por tecla e dois navegadores trocam eventos pelo Realtime.
1 — Bingo web (2 a 3 fins de semana)	Login anônimo, salas, lobby, tabuleiro, marcação em tempo real, timer, vitória, RLS e testes pgTAP.	4 pessoas jogam uma partida inteira pelo link, entrando em até 3 toques.
2 — App e clips (2 a 3 fins de semana)	App Tauri, assistente do OBS, clip por marcação, upload, galeria na sala.	Marcar uma casa gera um clip visível para todos em até 10 s em 3 jogos diferentes.
3 — Entrega e operação (1 a 2 fins de semana)	Vídeo final, limpeza automática, keepalive, release com Actions, atualização automática, assinatura.	Uma pessoa nova instala pelo link do GitHub e joga sem ajuda; limpeza comprovada em log.
4 — Dados (2 fins de semana)	Eventos, views, painel de estatísticas, biblioteca pública de objetivos.	Painel mostra a noite jogada e histórico por usuário.
5 — Evolução (opcional)	Versus, anti-bingo, marcação por conquistas da Steam, verificação de clip por visão, R2, modo sem OBS.	Cada item vira seu próprio ADR.

17. Engenharia: repositório, qualidade e entrega

17.1 Estrutura do repositório

```
bingoclip/
  apps/
    web/              # React + Vite + TS (UI do bingo)
    desktop/          # Tauri v2: shell nativo + módulos (obs, uploader, reel)
  packages/
    shared/           # tipos, regras de linhas/vitória, cliente Supabase
  supabase/
    migrations/       # SQL versionado (tabelas, RLS, RPCs)
    seed.sql          # biblioteca inicial de objetivos
    functions/        # purge-expired (Edge Function)
    tests/            # pgTAP: marcação, vitória, permissões
  docs/               # este documento, ADRs, guia do OBS, página de privacidade
  .github/workflows/ # ci.yml, release.yml, keepalive.yml
```

17.2 Qualidade

- **Banco:** testes pgTAP para marcação concorrente, vitória, expiração e permissões (um usuário não lê sala alheia).
- **Front:** Vitest para a lógica compartilhada e Playwright para o fluxo de entrada do convidado.
- **Ambientes:** dois projetos gratuitos do Supabase (dev e prod), o que usa exatamente o limite de 2 projetos ativos. Supabase local por Docker é opcional, pois consome bastante RAM.

- **Convenções:** migrations apenas para frente, commits convencionais, um ADR por decisão relevante em `docs/adr`.

17.3 CI/CD (Actions)

- **ci.yml:** lint, testes, build do web a cada PR. O Cloudflare Pages publica o front direto do repositório.
- **release.yml:** a cada tag, o `tauri-action` gera o instalador do Windows, publica nas Releases com o `latest.json` e o hash SHA-256. Repositório público mantém tudo gratuito.
- **keepalive.yml:** a cada 12 h, mantém o projeto ativo e dispara a limpeza.

```
name: keepalive
on:
  schedule: [{ cron: "17 */12 * * *" }]
  workflow_dispatch:
jobs:
  maintenance:
    runs-on: ubuntu-latest
    steps:
      - run: |
          curl -fsS -X POST "$SUPABASE_URL/rest/v1/rpc/ping" \
            -H "apikey: $SUPABASE_ANON_KEY" -H "Content-Type: application/json" -d '{}'
          curl -fsS -X POST "$SUPABASE_URL/functions/v1/purge-expired" \
            -H "Authorization: Bearer $SUPABASE_SERVICE_KEY"
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_ANON_KEY: ${{ secrets.SUPABASE_ANON_KEY }}
          SUPABASE_SERVICE_KEY: ${{ secrets.SUPABASE_SERVICE_KEY }}
```

17.4 Ambiente de desenvolvimento

- Node 20 ou superior e pnpm; Rust (rustup) e as dependências do Tauri para Windows (Build Tools do Visual Studio e WebView2).
- OBS Studio 28 ou superior e ffmpeg para os testes de clip.
- CLI do Supabase para migrations e testes; conta gratuita no Supabase, no Cloudflare e no GitHub.

18. Backlog inicial do MVP

- US-01. Como anfitrião, crio uma sala com tamanho, duração, linhas-alvo e política de clips, e recebo um link.
- US-02. Como convidado, entro pelo link com um apelido, sem criar conta.
- US-03. Como membro, vejo quem está na sala e quem tem clips ativos.
- US-04. Como anfitrião, escolho jogos e objetivos (ou escrevo os meus) e inicio a partida.
- US-05. Como jogador, marco uma casa e todos veem na hora quem marcou.
- US-06. Como grupo, vencemos ao completar as linhas-alvo antes do tempo acabar.

- US-07. Como jogador, instalo o app e o assistente verifica o OBS e grava um clip de teste.
- US-08. Como jogador com app, cada marcação gera um clip que aparece na galeria da sala.
- US-09. Como editor, gero o vídeo final com os melhores momentos e o salvo no meu PC.
- US-10. Como usuário, vinculo o Discord para guardar meu histórico e posso excluir meus dados.
- US-11. Como mantenedor, o sistema apaga clips e dados vencidos sem intervenção.
- US-12. Como mantenedor, vejo estatísticas da noite em um painel.

Definição de pronto (DoD) para cada história

- Migration e política de segurança revisadas, com teste automatizado quando envolver regra de jogo.
- Testado com pelo menos duas pessoas reais em máquinas diferentes.
- Nenhum novo serviço pago ou dependência fora dos limites gratuitos.
- ADR criado se houver decisão relevante.

19. Glossário e próximos passos

Termo	Significado
Replay Buffer	Recurso do OBS que mantém os últimos segundos de vídeo em memória e salva sob comando.
obs-websocket	Interface WebSocket do OBS para controlá-lo por programa; embutida no OBS 28+.
RLS	Row Level Security: o Postgres decide por linha o que cada usuário pode ver ou alterar.
RPC	Função no banco chamada pelo cliente; aqui concentra as regras do jogo.
Sidecar	Programa auxiliar executado pelo app (aqui, o ffmpeg).
Deep link	Link com esquema próprio (bingoclip://) que abre o app já na sala certa.
Egress	Tráfego de saída de dados do provedor; é o que o plano gratuito limita.

Próximos passos recomendados

1. Rodar o spike S1 (script que salva o replay por tecla) com o seu OBS e o jogo que você mais usa.
2. Criar o repositório público, os dois projetos Supabase em São Paulo e o esqueleto das migrations (Fase 0).
3. Implementar a Fase 1 inteira antes de qualquer código de OBS, para ter algo jogável cedo.

20. Fontes consultadas (10/10/2026)

- Supabase — preços e limites do plano gratuito: https://supabase.com/pricing e https://supabase.com/docs/pricing
- Supabase — limites do Realtime: https://supabase.com/docs/guides/realtime/limits
- Supabase — Auth (OAuth social, incluindo Discord): https://supabase.com/docs/guides/auth
- Resumos do plano gratuito do Supabase (pausa, egress, login anônimo): https://freetier.co/directory/products/supabase e https://aiagencyplus.com/supabase-free-tier-limits/
- OBS — obs-websocket incluído no OBS 28+: https://github.com/theca11/obs-websocket
- OBS — Replay Buffer (tamanho e memória): https://clip.dor.gg/en/blog/obs-replay-buffer-highlights e https://obsproject.com/forum/threads/replay-buffer-setting-missing.86417/
- Tauri vs Electron: https://blog.openreplay.com/comparing-electron-tauri-desktop-applications/ e https://capgo.app/blog/electron-vs-tauri-capacitor/
- Assinatura gratuita para código aberto (SignPath): https://super-productivity.com/code-signing/ e https://github.com/amd/gaia/issues/732
- Cloudflare Pages — limites: https://developers.cloudflare.com/pages/platform/limits
- Cloudflare R2 — preços: https://developers.cloudflare.com:8443/r2/llms-full.txt
- GitHub Actions — preços: https://cicdcalculator.com/github-actions-free-tier e https://github.blog/changelog/2025-12-16-coming-soon-simpler-pricing-and-a-better-experience-for-github-actions/
- Tailscale — plano gratuito: https://www.ssdnodes.com/learn/is-tailscale-free-plan-limits
- Hamachi e Radmin VPN: https://downloads.silicon.co.uk/?p=29249 e https://www.security.org/vpn/radminvpn/
- Túneis da Cloudflare: https://abcsteps.com/blog/cloudflare-tunnel-expose-localhost-without-opening-a-port
- WebRTC (STUN/TURN): https://github.com/meefik/peerix

Limitações desta pesquisa: os números vêm de páginas públicas e de artigos de terceiros (alguns divergem entre si, como o número de usuários do Tailscale gratuito). Itens marcados como “a validar” dependem dos spikes da seção 15.2.